

MODULE 3 – UI & LAYOUT RESPONSIVE

1

1. Mise en page avancée

Stack (superposition) Stack permet de superposer des widgets les uns sur les autres. Utile pour les badges, les images avec texte superposé, les overlays.

```
Stack(
  alignment: Alignment.center,
  children: [
    Image.network('https://example.com/photo.jpg'),
    Positioned(
      bottom: 16,
      left: 16,
      child: Container(
        padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
        decoration: BoxDecoration(
          color: Colors.black.withOpacity(0.6),
          borderRadius: BorderRadius.circular(8),
        ),
        child: const Text(
          'Nouveau',
          style: TextStyle(color: Colors.white, fontWeight: FontWeight.bold),
        ),
      ),
    ),
  ],
)
```

Positioned (ne fonctionne que dans un Stack) :

- top, bottom, left, right : positionnement absolu
- width, height : dimensions fixes

Expanded et Flexible

Dans une Row ou une Column, ces widgets distribuent l'espace restant.

```
Row(
  children: [
    Expanded(
      flex: 2, // prend 2 parts sur 3
      child: Container(color: Colors.red),
    ),
    Expanded(
      flex: 1, // prend 1 part sur 3
      child: Container(color: Colors.blue),
    ),
  ],
)
```

Widget	Comportement
Expanded	Remplit tout l'espace disponible (flex optionnel)
Flexible	Peut rétrécir si nécessaire (fit: FlexFit.loose)
Spacer	Équivalent à Expanded vide, pour créer des espaces



Règle d'or : si vous avez un Row ou Column qui déborde (overflow), enveloppez les enfants problématiques dans Expanded ou Flexible.

Wrap (retour à la ligne automatique) Quand vous ne savez pas combien d'éléments vous avez et que Row déborderait.

```
Wrap(
  spacing: 8,           // espace horizontal
  runSpacing: 8,       // espace vertical
  children: [
    Chip(label: Text('Flutter')),
    Chip(label: Text('Dart')),
    Chip(label: Text('Mobile')),
    Chip(label: Text('UI/UX')),
    Chip(label: Text('Firebase')),
  ],
)
```

2. Listes optimisées

ListView.builder (la liste professionnelle) Contrairement à ListView simple qui construit tous les enfants en mémoire, ListView.builder ne construit que les éléments visibles à l'écran. Indispensable pour les longues listes.

```
ListView.builder(
  itemCount: taches.length,
  itemBuilder: (context, index) {
    final tache = taches[index];
    return CarteTache(
      titre: tache.titre,
      categorie: tache.categorie,
      priorite: tache.priorite,
      avancement: tache.avancement,
    );
  },
)
```

Propriétés importantes :

- padding : marge intérieure de la liste
- scrollDirection : Axis.vertical (défaut) ou Axis.horizontal

- physics : comportement du scroll (BouncingScrollPhysics, ClampingScrollPhysics)
- separatorBuilder (ListView.separated) : ajoute un séparateur entre chaque élément

```
ListView.separated(
  itemCount: taches.length,
  separatorBuilder: (context, index) => const Divider(height: 1),
  itemBuilder: (context, index) => CarteTache(...),
)
```

GridView Pour afficher des éléments en grille (2 colonnes, 3 colonnes, etc.).

```
GridView.builder(
  gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,           // 2 colonnes
    crossAxisSpacing: 16,        // espace horizontal
    mainAxisSpacing: 16,         // espace vertical
    childAspectRatio: 0.75,      // ratio largeur/hauteur
  ),
  itemCount: taches.length,
  itemBuilder: (context, index) => CarteTacheCompacte(tache: taches[index]),
)
```

Types de GridDelegate :

- SliverGridDelegateWithFixedCrossAxisCount : nombre fixe de colonnes
- SliverGridDelegateWithMaxCrossAxisExtent : colonnes adaptatives selon la largeur disponible

3. Responsive Design

MediaQuery Accède aux informations sur l'écran : taille, orientation, densité de pixels, padding système.

```
final size = MediaQuery.of(context).size;
final largeur = size.width;
final hauteur = size.height;
final estTablette = largeur > 600;
final orientation = MediaQuery.of(context).orientation;
```

Cas d'usage courants :

```
// Taille relative
Container(width: largeur * 0.9)

// Padding adaptatif au notch/status bar
Padding(padding: MediaQuery.of(context).padding)

// Texte adaptatif
Text('Titre', style: TextStyle(fontSize: largeur * 0.05))
```

LayoutBuilder Reçoit les contraintes de son parent et s'adapte. Plus performant que MediaQuery pour les décisions de layout.

```
LayoutBuilder(
```

```
builder: (context, constraints) {
  if (constraints.maxWidth > 900) {
    return VueTabletteDesktop(); // 3 colonnes
  } else if (constraints.maxWidth > 600) {
    return VueTablette(); // 2 colonnes
  } else {
    return VueMobile(); // 1 colonne
  }
},
)
```

MediaQuery vs LayoutBuilder :

- MediaQuery : taille de l'écran entier. Utile pour les décisions globales.
- LayoutBuilder : taille disponible pour CE widget. Utile pour les widgets réutilisables.

OrientationBuilder Réagit aux changements d'orientation (portrait / paysage).

```
OrientationBuilder(
  builder: (context, orientation) {
    return GridView.count(
      crossAxisCount: orientation == Orientation.portrait ? 2 : 4,
      children: [...],
    );
  },
)
```

Breakpoints recommandés (Material Design)

Appareil	Largeur	Adaptation
Mobile	< 600 dp	1 colonne, pleine largeur
Tablette	600 –900 dp	2 colonnes, marges plus larges
Desktop	> 900 dp	3+ colonnes, drawer permanent

4. Thème global et Material Design 3

ThemeData Définit les couleurs, typographies, formes et comportements par défaut de toute l'application.

```
MaterialApp(
  theme: ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(
      seedColor: const Color(0xFF6750A4),
      brightness: Brightness.light,
    ),
    textTheme: const TextTheme(
      displayLarge: TextStyle(fontSize: 32, fontWeight: FontWeight.bold),
      titleLarge: TextStyle(fontSize: 22, fontWeight: FontWeight.w600),
      bodyLarge: TextStyle(fontSize: 16, height: 1.5),
    ),
  ),
)
```

```

    ),
    cardTheme: CardTheme(
      elevation: 2,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
    ),
  ),
  home: const PageAccueil(),
)

```

Accéder au thème dans n'importe quel widget :

```

final theme = Theme.of(context);
final colors = theme.colorScheme;

Text('Titre', style: theme.textTheme.titleLarge)
Container(color: colors.primary)
Icon(Icons.star, color: colors.onPrimary)

```

Material Design 3 La dernière version du système de design de Google. En Flutter, activez-le avec `useMaterial3: true`.

Composants M3 clés :

- `NavigationBar` (bottom bar moderne)
- `NavigationDrawer` (drawer avec sections)
- `FloatingActionButton` (3 tailles : small, regular, large)
- `Card` (3 élévations : elevated, filled, outlined)
- `Chip` (assist, filter, input, suggestion)

Gestion des assets Images

```

# pubspec.yaml

flutter:
  assets:
    - assets/images/
    - assets/images/logo.png
    - assets/icons/

Image.asset('assets/images/logo.png')

```

Polices personnalisées

```

flutter:
  fonts:
    - family: Roboto
      fonts:
        - asset: assets/fonts/Roboto-Regular.ttf
        - asset: assets/fonts/Roboto-Bold.ttf
      weight: 700

const Text('Titre', style: TextStyle(fontFamily: 'Roboto', fontWeight: FontWeight.bold))

```

✂ ÉNONCÉS DES TRAVAUX PRATIQUES

TP1 – Liste principale avec ListView.builder (1h)

Contexte Vous enrichissez l'écran d'accueil du fil rouge. Les données factices du Module 2 deviennent une vraie liste scrollable avec des cartes professionnelles.

Objectifs pédagogiques

- Maîtriser ListView.builder et ListView.separated
- Créer un widget CarteTache réutilisable et bien stylisé
- Utiliser Expanded, Padding, Card de manière professionnelle
- Comprendre l'intérêt de const dans les widgets statiques

Consignes

Étape 1 – Améliorer le modèle Tache Dans lib/models/tache.dart, enrichissez la classe

:

```
class Tache {
  final String id;
  final String titre;
  final String description;
  final String categorie;
  final String priorite; // 'Haute', 'Moyenne', 'Basse'
  final double avancement; // 0.0 à 1.0
  final DateTime dateEcheance;

  const Tache({
    required this.id,
    required this.titre,
    this.description = "",
    required this.categorie,
    required this.priorite,
    required this.avancement,
    required this.dateEcheance,
  });
}
```

Étape 2 – Créer le widget CarteTache amélioré Dans lib/widgets/carte_tache.dart, créez un StatelessWidget qui

Spécifications techniques :

- Card avec elevation: 2, margin: EdgeInsets.symmetric(horizontal: 16, vertical: 8)
- Padding intérieur de 16
- Column principale avec crossAxisAlignment: CrossAxisAlignment.start
- Première Row :
- Container circulaire de 12px de diamètre, couleur selon priorité (rouge/orange/vert)
- SizedBox(width: 12)
- Expanded avec le titre (theme.textTheme.titleMedium, bold)

- Chip avec la catégorie (petit, fond gris clair)
- SizedBox(height: 8)
- Description avec maxLines: 2, overflow: TextOverflow.ellipsis
- SizedBox(height: 12)
- LinearProgressIndicator avec borderRadius, couleur selon avancement :
- < 30% : rouge
- 30-70% : orange
- 70% : vert
- SizedBox(height: 4)
- Texte du pourcentage aligné à droite
- SizedBox(height: 12)
- Dernière Row : icône calendrier + date formatée (DateFormat du package intl) + icône flèche

Étape 3 – Mettre à jour main.dart

- Remplacez la liste statique par un ListView.builder
- Utilisez 8 tâches factices (au lieu de 5)
- Ajoutez un ListView.separated avec un Divider entre les cartes
- Le AppBar doit avoir un bottom avec un TabBar ou un PreferredSize contenant des statistiques (nombre de tâches, tâches en retard)

Livrable attendu : Code source + captures d'écran (portrait et paysage).

TP2 – Grille responsive avec GridView (45 min)

Contexte Sur tablette ou en paysage, la liste verticale gaspille de l'espace. Vous ajoutez une vue en grille qui s'active automatiquement.

Objectifs pédagogiques

- Utiliser GridView.builder
- Combiner LayoutBuilder pour les décisions de layout
- Créer un widget CarteTacheCompacte pour la grille

Consignes Étape 1 – Créer CarteTacheCompacte Dans lib/widgets/carte_tache_compacte.dart, créez une version allégée de la carte :

- Pas de description
- Titre en 1 ligne (ellipsis)
- Priorité sous forme de badge coloré en haut à droite
- Avancement sous forme de cercle (CircularProgressIndicator) au centre

- Catégorie en bas

Étape 2 – Implémenter le responsive dans PageAccueil Utilisez LayoutBuilder pour décider :

- Si maxWidth < 600 : ListView.builder (vue mobile)
- Si maxWidth >= 600 : GridView.builder avec crossAxisCount: 2 (vue tablette)
- Si maxWidth >= 900 : GridView.builder avec crossAxisCount: 3 (vue desktop)

Étape 3 – Tester

- Lancez sur un émulateur téléphone (portrait)
- Passez en paysage
- Lancez sur un émulateur tablette (10 pouces)
- Vérifiez que la transition est fluide

Livrable attendu : Code source + captures des 3 configurations.

TP3 – Thème cohérent et personnalisé (45 min)

Contexte Votre application utilise les couleurs par défaut de Flutter. Vous allez créer un thème professionnel cohérent avec la charte graphique du projet.

Objectifs pédagogiques

- Configurer ThemeData global
- Utiliser ColorScheme.fromSeed
- Personnaliser TextTheme et CardTheme
- Préparer le terrain pour le mode sombre (Module 9)

Consignes Étape 1 – Définir la palette de couleurs Dans lib/core/theme.dart, créez un fichier dédié :

```
import 'package:flutter/material.dart';

class AppTheme {
  static const Color couleurPrimaire = Color(0xFF6750A4);
  static const Color couleurSecondaire = Color(0xFF625B71);
  static const Color couleurTertiaire = Color(0xFF7D5260);

  static ThemeData themeClair = ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(
      seedColor: couleurPrimaire,
      brightness: Brightness.light,
    ),
    textTheme: _textTheme,
    cardTheme: _cardTheme,
```



```

    appBarTheme: _appBarTheme,
  );

  static const TextTheme _textTheme = TextTheme(
    displayLarge: TextStyle(fontSize: 32, fontWeight: FontWeight.bold, letterSpacing: -0.5),
    titleLarge: TextStyle(fontSize: 22, fontWeight: FontWeight.w600),
    bodyLarge: TextStyle(fontSize: 16, height: 1.5),
    bodyMedium: TextStyle(fontSize: 14, height: 1.4),
    labelLarge: TextStyle(fontSize: 14, fontWeight: FontWeight.w600, letterSpacing: 0.5),
  );

  static final CardTheme _cardTheme = CardTheme(
    elevation: 2,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
    margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
  );

  static const AppBarTheme _appBarTheme = AppBarTheme(
    elevation: 0,
    centerTitle: true,
  );
}

```

Étape 2 – Appliquer le thème Dans main.dart :

```

MaterialApp(
  title: 'Gestion de Tâches',
  theme: AppTheme.themeClair,
  home: const PageAccueil(),
);

```

Étape 3 – Utiliser le thème dans les widgets Dans CarteTache, remplacez toutes les couleurs et styles en dur par :

- Theme.of(context).colorScheme.primary • Theme.of(context).textTheme.titleMedium
- etc.

Étape 4 – Ajouter une police personnalisée

- Téléchargez une police Google Fonts (ex. Roboto, Inter, Poppins)
- Placez-la dans assets/fonts/
- Déclarez-la dans pubspec.yaml
- Utilisez-la dans TextTheme

Livrable attendu : Code source + captures montrant la cohérence visuelle.

FICHE RÉCAP STAGIAIRE – UI & Layout Responsive

10

Points clés à retenir

Concept	Quand l'utiliser ?
ListView.builder	Listes longues (plus de 10 éléments). Performant.
ListView.separated	Listes avec séparateurs visuels (dividers).
GridView.builder	Grilles d'éléments (catalogue, galerie).
Expanded	Remplir l'espace restant dans Row/Column.
Flexible	Même chose mais peut rétrécir si besoin.
Wrap	Liste d'éléments qui retourne à la ligne automatiquement.
Stack	Superposition de widgets (badges, overlays).
MediaQuery	Connaître la taille totale de l'écran.
LayoutBuilder	Adapter le layout selon l'espace disponible pour CE widget.
OrientationBuilder	Réagir au changement portrait/paysage.

Breakpoints responsive

```
// Dans LayoutBuilder
if (constraints.maxWidth < 600) {
  return VueMobile();           // 1 colonne
} else if (constraints.maxWidth < 900) {
  return VueTablette();         // 2 colonnes
} else {
  return VueDesktop();          // 3+ colonnes
}
```

Accéder au thème

```
final theme = Theme.of(context);
final colors = theme.colorScheme;
final textStyles = theme.textTheme;

// Couleurs
colors.primary           // Couleur principale
colors.onPrimary         // Texte sur couleur principale
colors.secondary         // Couleur secondaire
colors.surface           // Fond des cartes
colors.error             // Rouge d'erreur

// Textes
textStyles.displayLarge // Très grand titre
textStyles.titleLarge   // Titre de page
textStyles.bodyLarge    // Corps de texte principal
textStyles.labelLarge   // Boutons, labels
```

Erreurs fréquentes à éviter

1. **Utiliser ListView simple avec 100+ éléments** → crash mémoire. Utilisez ListView.builder.
2. **Oublier Expanded dans une Row avec Text long** → overflow. Enveloppez le Text dans Expanded.
3. **Hardcoder les couleurs** → thème impossible à changer. Utilisez toujours Theme.of(context).
4. **Ne pas tester en paysage** → 30% des utilisateurs tournent leur téléphone.
5. **GridView sans childAspectRatio** → cartes trop hautes ou trop larges. Ajustez ce ratio.
6. **Oublier SafeArea** → contenu caché par le notch ou la barre de navigation.

Check mentale avant de coder un écran complexe

- ☐ La liste est-elle longue ? → ListView.builder
- ☐ Les éléments débordent-ils ? → Expanded / Flexible / Wrap
- ☐ Dois-je superposer des éléments ? → Stack + Positioned
- ☐ L'écran doit-il s'adapter ? → MediaQuery / LayoutBuilder
- ☐ Les couleurs sont-elles dans le thème ? → Theme.of(context)
- ☐ Ai-je testé portrait ET paysage ? → Émulateur
- ☐ Le contenu est-il dans une SafeArea ? → Pas de contenu caché

QUIZ DE VALIDATION – Module 3

Durée : 10 minutes – 8 questions

Question 1 (QCM)

Quel widget permet d'afficher une liste de 1000 éléments sans saturer la mémoire ?

- A. ListView
- B. ListView.builder
- C. Column
- D. SingleChildScrollView

Question 2 (QCM)

Dans un Stack, quel widget permet de positionner un enfant à des coordonnées précises ?

- A. Align
- B. Positioned
- C. Container
- D. SizedBox

Question 3 (Ouvrte)

Quelle est la différence entre MediaQuery et LayoutBuilder ? Donnez un cas d'usage pour chacun.

Réponse attendue :

- MediaQuery donne la taille de l'écran entier. Utile pour décider du layout global (ex. drawer permanent sur desktop).
- LayoutBuilder donne les contraintes du widget parent. Utile pour les widgets réutilisables qui s'adaptent à leur espace disponible.

Question 4 (QCM)

Quelle propriété de SliverGridDelegateWithFixedCrossAxisCount définit le nombre de colonnes ?

- A. crossAxisCount
- B. mainAxisCount
- C. columnCount
- D. itemCount

Question 5 (Ouverte)

Écrivez le code d'un LayoutBuilder qui affiche un Text('Mobile') si la largeur est inférieure à 600, sinon un Text('Tablette').

Réponse attendue :

```
LayoutBuilder(
  builder: (context, constraints) {
    if (constraints.maxWidth < 600) {
      return const Text('Mobile');
    } else {
      return const Text('Tablette');
    }
  },
)
```

Question 6 (QCM)

Pour utiliser une image locale assets/images/logo.png, quelle étape est obligatoire ?

- A. Importer le package image_picker
- B. Rien, Flutter détecte automatiquement
- C. Déclarer le chemin dans pubspec.yaml sous flutter > assets
- D. Convertir l'image en base64

Question 7 (QCM)

Quel widget crée un espace flexible entre deux enfants d'une Row ?

- A. SizedBox.expand
- B. Padding
- C. Spacer
- D. Divider

Question 8 (QCM – difficulté +)

Que se passe-t-il si vous placez un Text très long dans une Row sans Expanded ?

- A. Le texte est automatiquement tronqué
- B. Le texte défile horizontalement

- C. Une erreur d'overflow s'affiche (bande jaune/noire)
- D. La Row s'agrandit pour tout contenir

Barème indicatif

- 8/8 : Prêt pour le Module 4 (Navigation)
 - 6-7/8 : Acquis solides, réviser LayoutBuilder vs MediaQuery
 - 4-5/8 : Refaire les TP, le Module 4 sera difficile
 - < 4/8 : Revoir la théorie responsive et les widgets de layout
-

ANNEXE – Ressources et liens

- **Widget catalog – Layout** : <https://docs.flutter.dev/ui/widgets/layout>
- **Responsive design** : <https://docs.flutter.dev/ui/layout/responsive>
- **Material Design 3** : <https://m3.material.io>
- **Google Fonts Flutter** : https://pub.dev/packages/google_fonts
- **Intl (dates)** : <https://pub.dev/packages/intl>